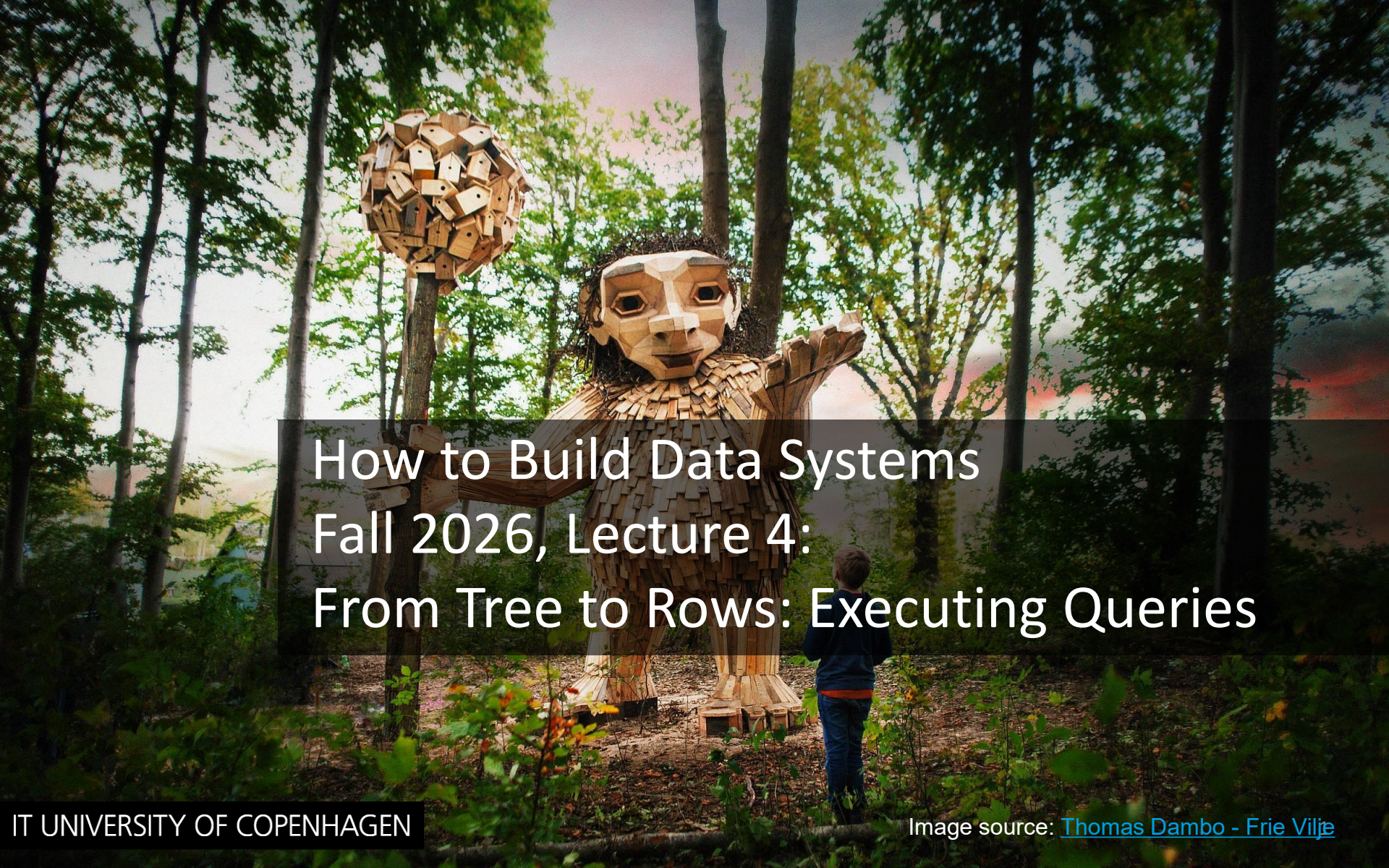




How to Build Data Systems Fall 2026, Lecture 4: From Tree to Rows: Executing Queries

Stand-Up & Exercise Recap

From Tree to Rows: Executing Queries

- Operator Tree
- Volcano vs. Other Execution Models
- Planning
- Executing

2-Minute Stand-Up

Summarize your parser and binder implementation.

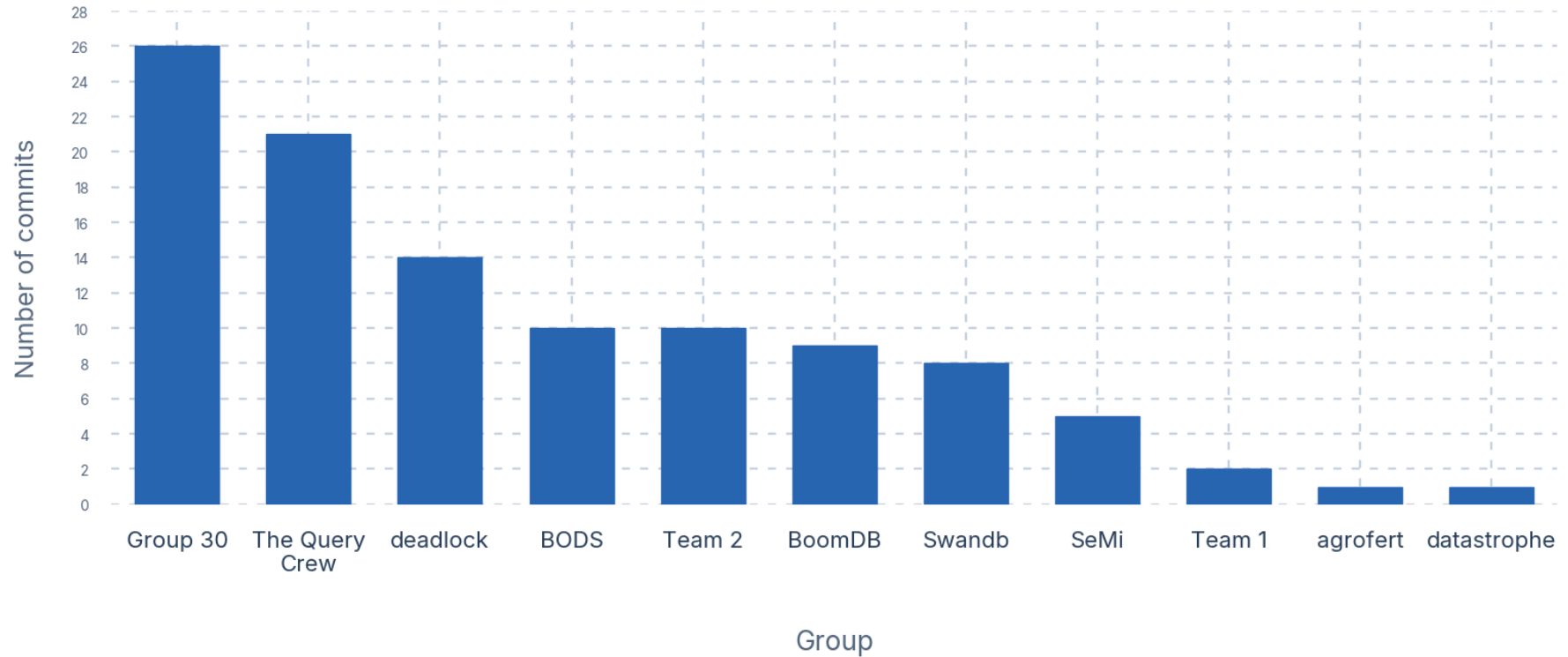
What errors are you logging?

What problems did you encounter during your implementation?

Team Progress

Commits by group

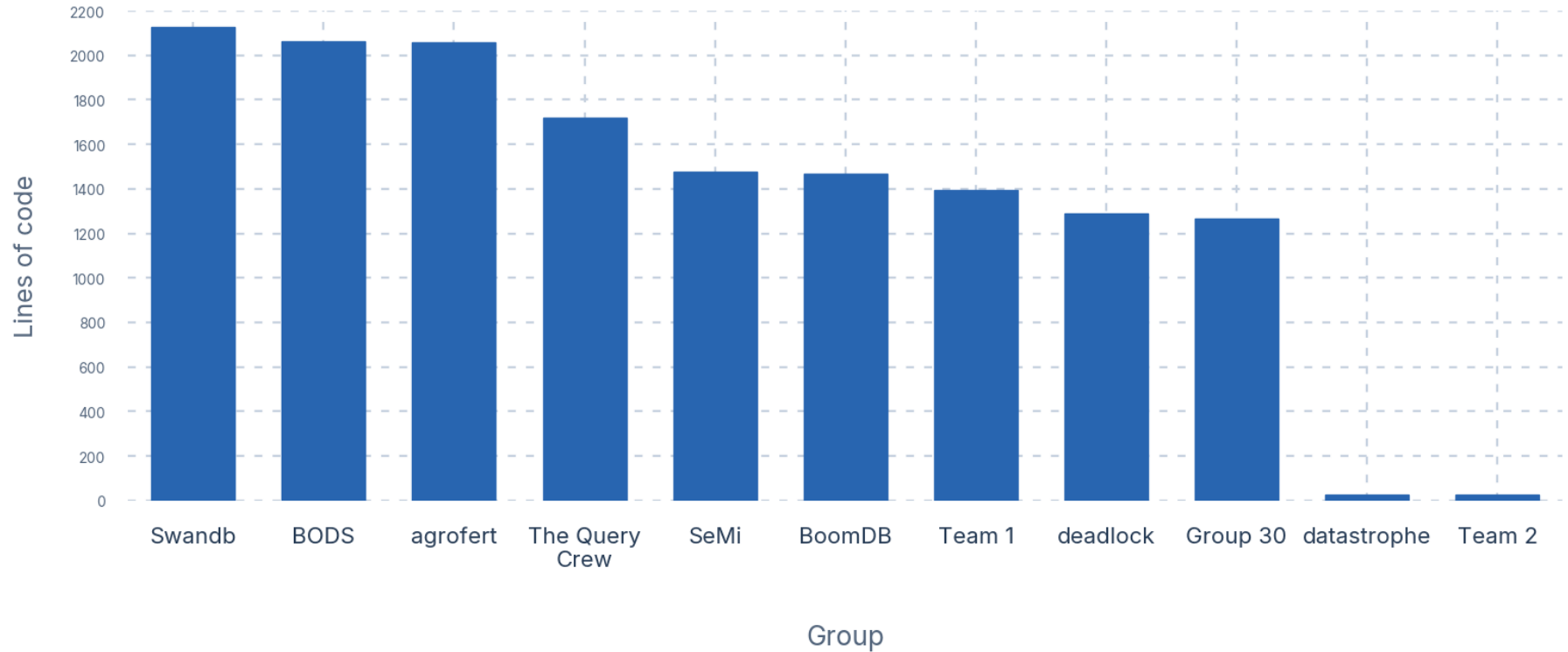
10–16 September 2026



Team Progress

Lines of code by group

16/09/26 · Lines of code without comments



Exercise 3 Parsing Good and Bad Queries

Group	Parse + AST /77	Pretty-print /77	Good complete /77	Bad correctly rejected /87	Status
agrofert	77	72	72	87	Tested
bods	77	0	0	87	Tested
BoomDB	77	72	72	87	Tested
datastrophe	N/A	N/A	N/A	N/A	Not implemented
deadlock	77	72	72	87	Tested
group-30	77	72	72	87	Tested
SeMi	76	71	71	87	Tested
swandb	77	77	77	87	Tested
team1	77	77	77	87	Tested
team2	N/A	N/A	N/A	N/A	Not implemented
TheQueryCrew	77	75	75	87	Tested

Professor engine and student comparison

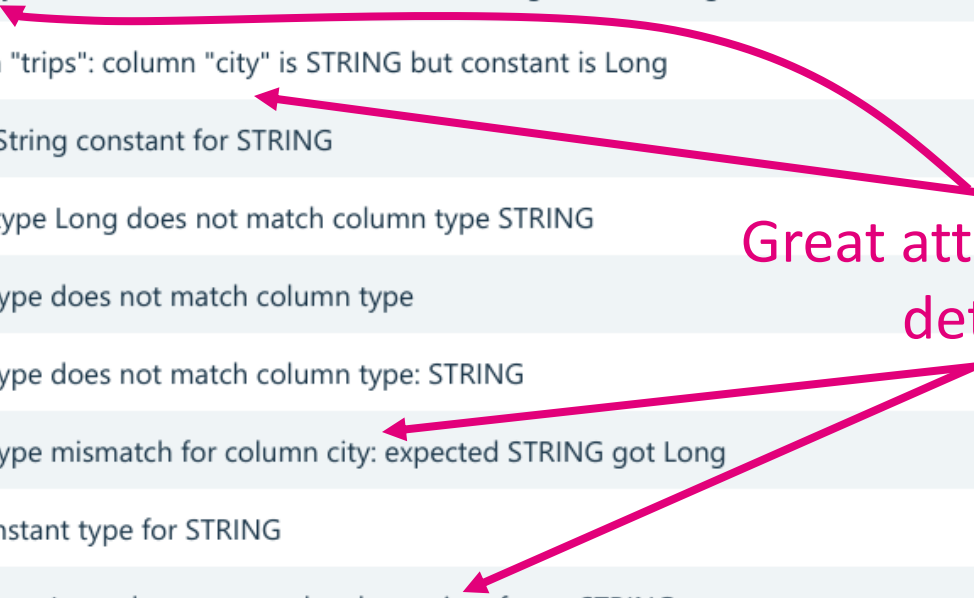
The added `datasys-claude-engine` repository was tested twice: its current local checkout and its `v0.3` tag. The latter is the direct Exercise 3 comparison. Both used the same 77 good and 87 bad inputs and the same strict printer check.

Professor version	Parse + AST /77	Good complete /77	Bad rejected /87	Required-stage rejection /87
v0.3	77	72	87	87
Current	77	72	86	83

Both professor versions fail G0036-G0040: extreme Doubles are printed using exponent notation, such as `4.9E-324`, which their own grammars reject. Both preserve negative zero and accept the empty COPY path. The current checkout is `1e682f27388c`; the `v0.3` release is `e7a5989f4701`.

Binder Error Messages

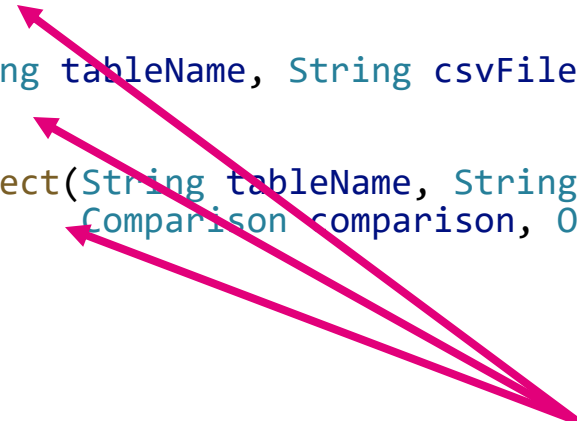
Group	Observed type-mismatch message (B0067)
agrofert	column city is STRING so the constant must be String but was Long
bods	SELECT on "trips": column "city" is STRING but constant is Long
BoomDB	expected String constant for STRING
deadlock	Constant type Long does not match column type STRING
group-30	constant type does not match column type
SeMi	constant type does not match column type: STRING
swandb	constant type mismatch for column city: expected STRING got Long
team1	Invalid constant type for STRING
TheQueryCrew	constant type Long does not match column city of type STRING



Great attention to detail

Exercise 2 Recap

```
public void createTable(String tableName, List<ColumnSpec> columns) { /* ... */ }  
  
public void copyFile(String tableName, String csvFilePath) { /* ... */ }  
  
public List<Object[]> select(String tableName, String columnName,  
                             Comparison comparison, Object constant) { /* ... */ }
```



No connection to
last week's SQL yet

Exercise 3 Recap

```
SELECT * FROM trips WHERE distance > 100;
```

SQL text



```
SqlParser.parse(sql)
```

```
List<Statement> statements
```

AST objects



```
Binder.bind(statements)
```

```
List<BoundStatement> boundStatements
```

AST objects



```
SqlPrinter.print(boundStatements)
```

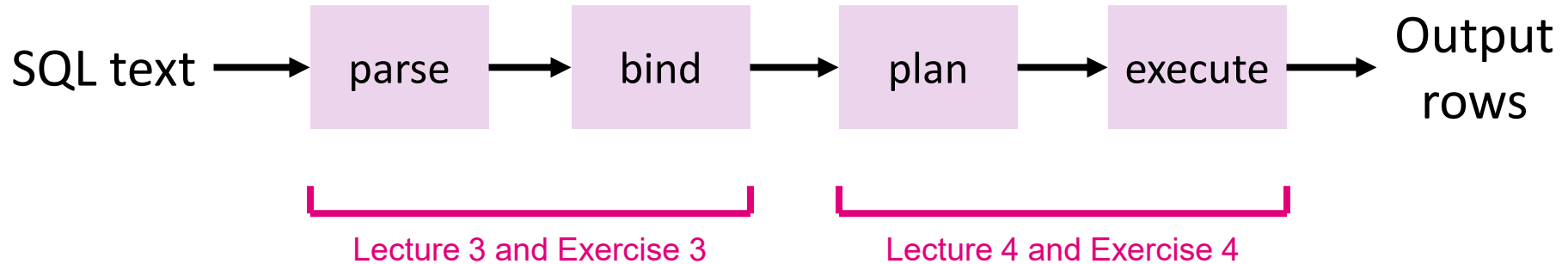
```
SELECT * FROM trips WHERE distance > 100;
```

SQL text

Exercise 4 Preview

The pipeline gets two new boxes: plan and execute

- Left half exists since last week
- Right half is this week's work



What Changes in the Code Base

New

- `Operator` interface, `ScanOperator`, `FilterOperator`
- `Planner` (pruning moves here), `Plan` record with `ScanStats`
- `Executor` running parse, bind, plan, execute

Changed

- `Engine.main` becomes the SQL front door (statement, script, usage)
- `select(...)` keeps its week 2 signature but plans and drains a pipeline internally → needs refactoring!

Unchanged

- Exercise 2 integration tests! When they are green, the refactor is done.
- Note: We're releasing exercises incrementally for you to practice refactoring. Critical skill in the industry!

Exercise 2 API

Exercise 2 API stays the API of the storage layer:

```
public void createTable(String tableName, List<ColumnSpec> columns)
public void copyFile(String tableName, String csvFilePath)
public List<Object[]> select(String tableName, String columnName,
                             Comparison comparison, Object constant)
```

- **CREATE TABLE** and **COPY** have no operator tree: the executor calls `createTable()` and `copyFile()` directly.
- Only **SELECT** gets an operator tree. Everything about operators today is about **SELECT**.
- `Select()` calls plan + execute. The API remains only for testing the refactor. Can be removed after v0.4 is tagged.

Exercise 2 API's select()

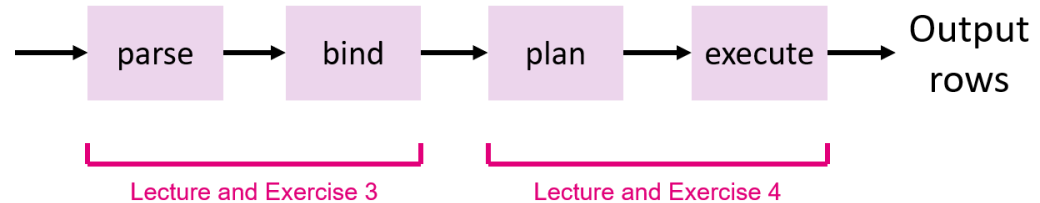
Exercise 2's select()

```
public List<Object[]> select(  
    String tableName,  
    String columnName,  
    Comparison comparison,  
    Object constant) {  
  
    // prune partitions  
    // read partitions  
    // filter rows  
    // return rows  
  
}
```

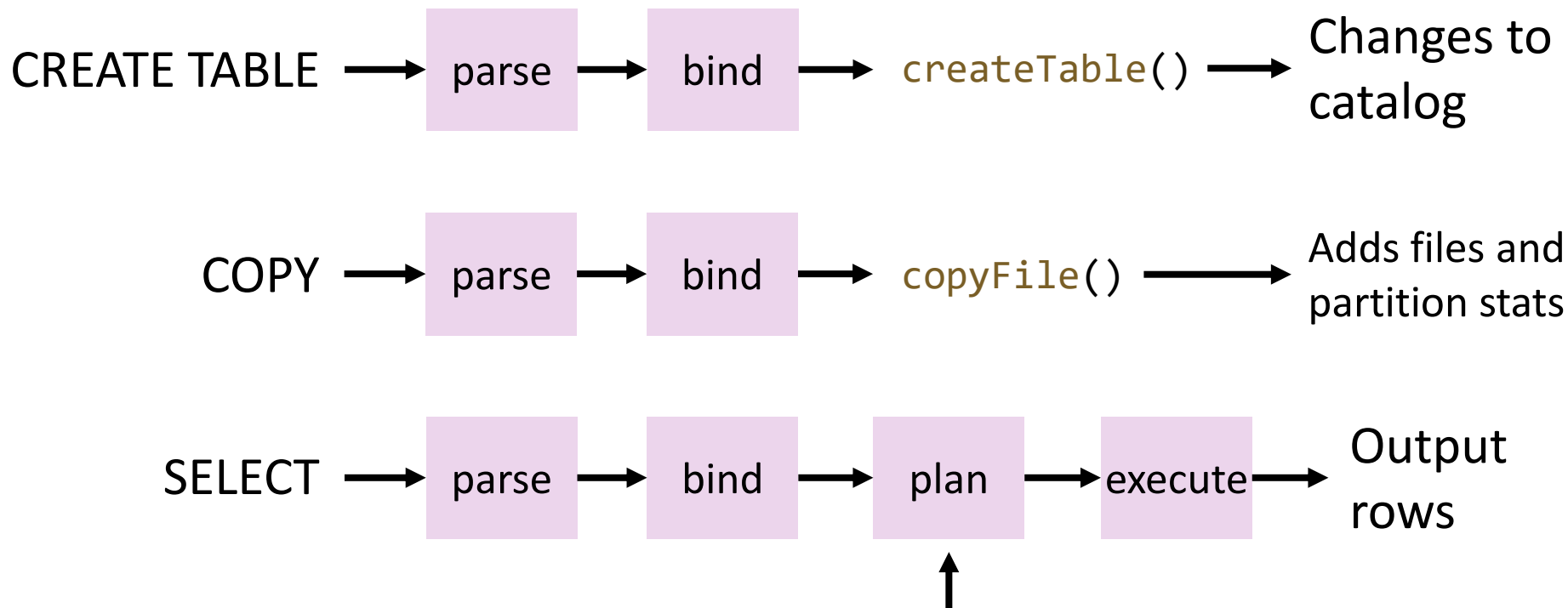
Same API
Same tests
Different implementation

Exercise 4's select()

```
public List<Object[]> select(  
    String tableName,  
    String columnName,  
    Comparison comparison,  
    Object constant) {  
  
    // call plan & execute  
    // return rows  
  
}
```



How Exercises 2, 3, 4 Connect



`select()` calls into `plan` (as on the previous slide)
Exercise 2 tests call `createTable()`, `copyFile()`, `select()`
Exercise 4 integration tests call the whole pipeline
If all tests are green, `select()` can be removed.

Stand-Up & Exercise Recap

From Tree to Rows: Executing Queries

- Operator Tree
- Volcano vs. Other Execution Models
- Planning
- Executing

From Trees to Rows

```
SELECT * FROM trips WHERE distance > 100;
```

What to compute vs. how to compute it

- Bound statement: what the query means
 - Table: trips
 - Filter: distance > 100
- Physical plan: how the engine computes it
 - Which partitions to open
 - Which operators
 - In which order



Example Table

Golden data from Exercise 2, sorted by distance, `maxRowsPerPartition = 2`

```
trips (city STRING, distance LONG, price DOUBLE);
```

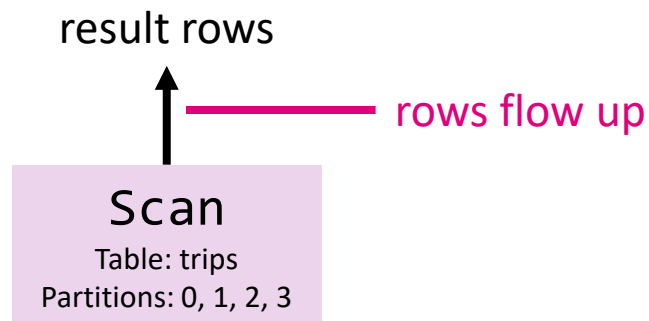
Partition 0	Copenhagen	12	23.5	Distance: min=12 max=31
	Roskilde	31	45.0	
Partition 1	Copenhagen	88	99.99	Distance: min=88 max=95
	Odense	95	120.75	
Partition 2	Copenhagen	140	210.0	Distance: min=140 max=187
	Aarhus	187	301.0	
Partition 3	Aalborg	210	340.5	Distance: min=210 max=299
	Esbjerg	299	450.25	

Physical Plan: Operator Tree

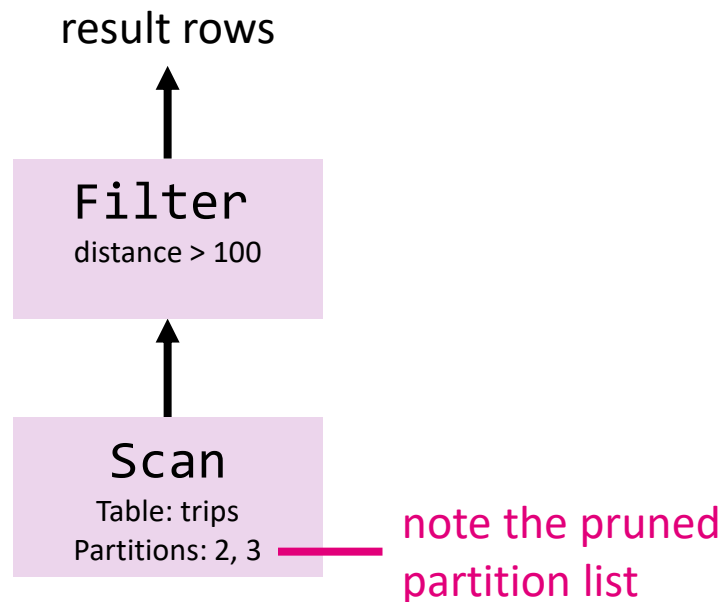
The Operator Tree for Our SELECT

- Each node produces rows, each parent consumes them

```
SELECT * FROM trips;
```

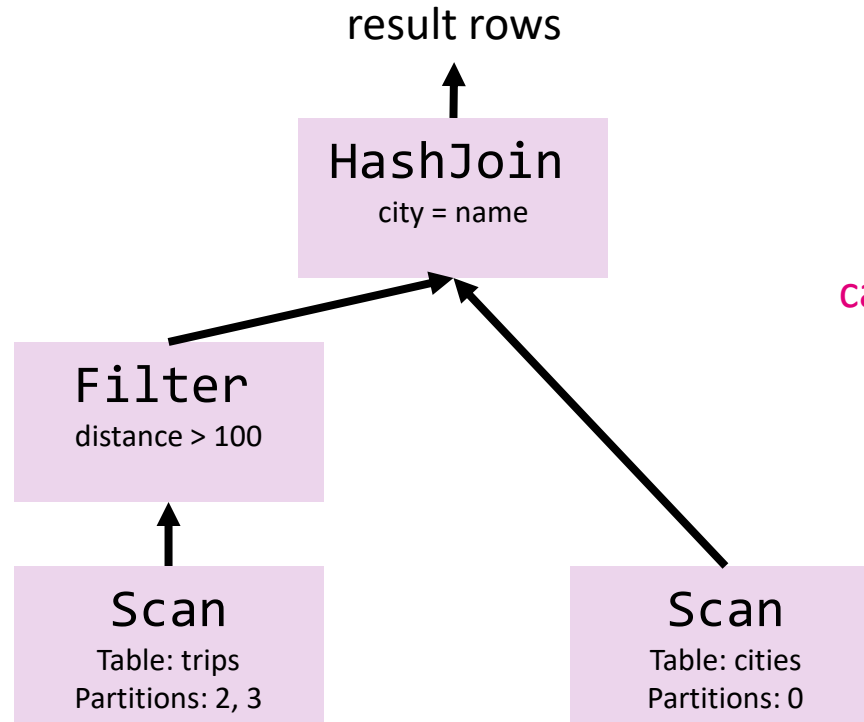


```
SELECT * FROM trips WHERE distance > 100;
```



It Really Is a Tree! (Wait for Part 3)

```
SELECT * FROM trips, cities WHERE city = name  
AND distance > 100;
```



In practice
operator trees
can become large
(1000+ nodes)

Operator Interface

```
public interface Operator {  
    void open();          // prepare; called once before the first next()  
    Object[] next();      // one row in schema column order, null when exhausted  
    void close();         // release what was held; write the summary log line  
}
```

Every operator implements the same three methods:

- **open()**: Allocates resources, initializes internal state, and prepares the operator to produce data.
- **next()**: Fetches exactly one tuple at a time. It returns the tuple to the calling operator above it or **null** indicating that execution is complete.
- **close()**: Cleans up state, releases memory, and closes any open files or underlying operators.

History of the Iterator Model

Goetz Graefe, [*Volcano - An Extensible and Parallel Query Evaluation System*](#), IEEE TKDE 6(1), 1994. (Optional reading)

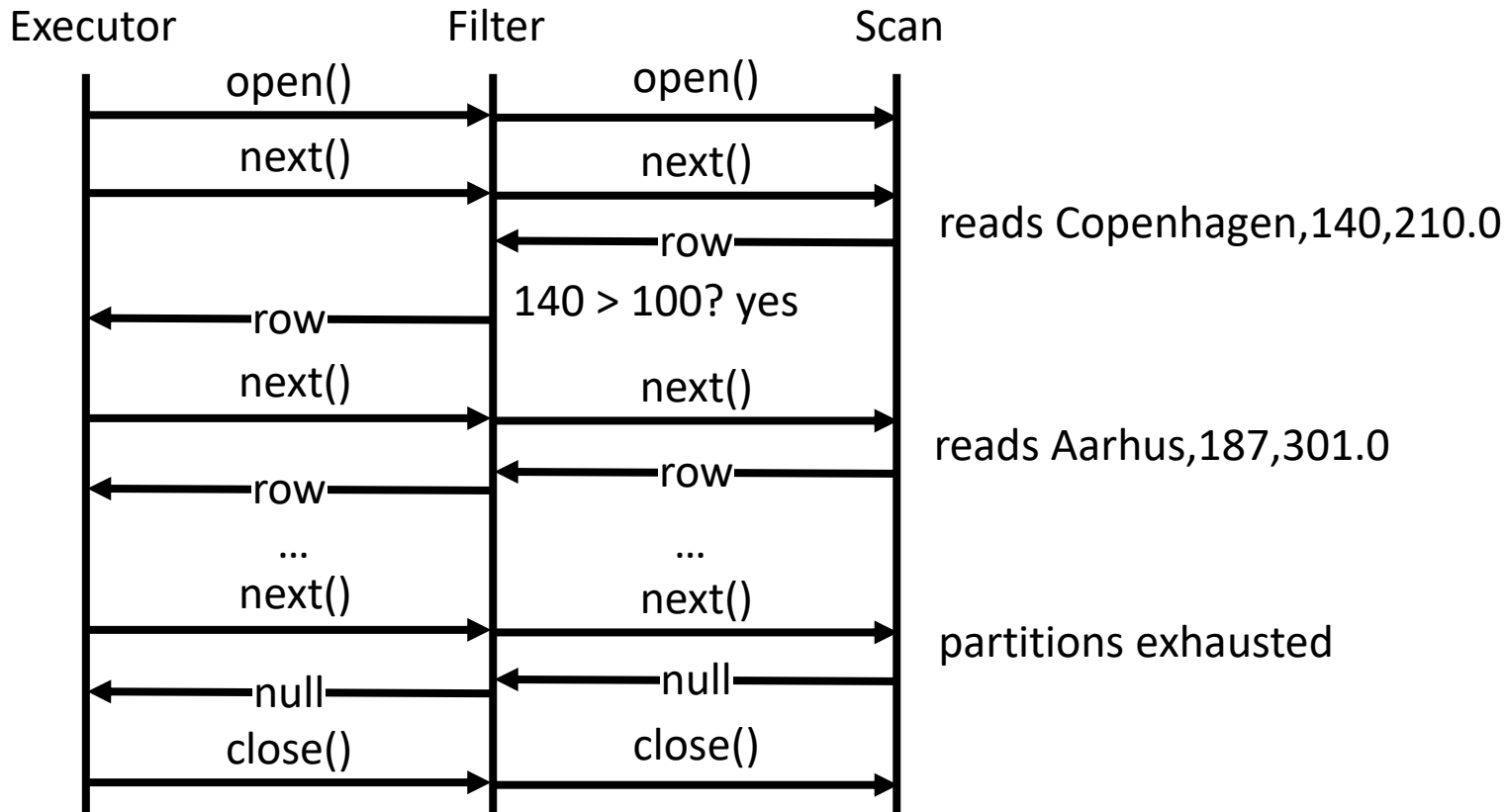
Two design goals:

1. Extensibility: The uniform iterator interface made operators composable: any operator can be the input of any other
2. Parallelism: Via the Exchange operator, the Volcano engine can handle high-performance parallel architectures without rewriting the core query operators (more in Lecture 9)

Still the default mental model: PostgreSQL, MySQL, SQLite execute this way.

Pull-Based Dataflow

```
SELECT * FROM trips WHERE distance > 100;
```



Example: Scan Operator Implementation

```
public final class ScanOperator implements Operator {
    private final StorageEngine engine;
    private final String tableName;
    private final List<Integer> partitionNumbers; // handed in by the planner
    private int nextPartition;
    private Iterator<Object[]> rows = Collections.emptyIterator();
    private int rowsOut;

    public void open() { nextPartition = 0; rows = Collections.emptyIterator(); rowsOut = 0; }

    public Object[] next() {
        while (!rows.hasNext()) {
            if (nextPartition == partitionNumbers.size()) return null;
            rows = engine.readPartition(tableName, partitionNumbers.get(nextPartition++)).iterator();
        }
        rowsOut++;
        return rows.next();
    }

    public void close() { rows = Collections.emptyIterator();
        LOGGER.debug("table={} partitions={} rowsOut={}", tableName, partitionNumbers.size(), rowsOut);
    }
}
```

pruned partition list

return null if all partitions are exhausted

read next partition if current partition is exhausted

Example: Filter Operator Implementation

```
public final class FilterOperator implements Operator {
    private final Operator child;
    private final ColumnPredicate predicate;
    private int rowsIn, rowsOut;

    public void open() { rowsIn = 0; rowsOut = 0; child.open(); }

    public Object[] next() {
        Object[] row = child.next();
        while (row != null) {
            rowsIn++;
            if (predicate.matches(row)) { rowsOut++; return row; }
            row = child.next();
        }
        return null;
    }

    public void close() { child.close(); LOGGER.debug("rowsIn={} rowsOut={}", rowsIn, rowsOut); }
}
```

next row from the child operator

only returning the row if it matches the filter predicate

returning null if no more rows from the child that match

Properties of Operators (1)

Memory Usage

Defines how much hardware RAM an operator needs to process data

- **Constant Memory $O(1)$:** Processes rows on the fly or maintains a fixed-size scalar state.
- **Linear Memory $O(N)$ / $O(K)$:** Scales with input size (N) or unique groups/keys (K). Requires buffering.

Spilling Risk: When data exceeds RAM, operators must gracefully “spill” (write) to disk, causing massive I/O performance penalties.

Properties of Operators (2)

Blocking Behavior (Pipeline Breakers)

Defines when an operator passes data to the next step in the query plan.

- **Non-Blocking:** Emits rows immediately to the next operator. Benefit: Low first-row latency.
- **Blocking:** Must consume 100% of its input before emitting its first output row. Penalty: Freezes downstream execution.

Quiz Time (1)

Fill out memory usage and blocking behavior for scan and filter operators

	Memory Usage	Blocking Behavior
Scan	$O(1)$, $O(K)$, $O(N)$?	Blocking or not?
Filter	$O(1)$, $O(K)$, $O(N)$?	Blocking or not?



Example: Project Operator Implementation

`SELECT city, price FROM trips;`

```
public final class ProjectOperator implements Operator {
    private final Operator child;
    private final int[] columns;    // for each output column, its index in the child's row
    private int rowsOut;

    public void open() { rowsOut = 0; child.open(); }

    public Object[] next() {
        Object[] row = child.next();
        if (row == null) return null;
        Object[] projected = new Object[columns.length];
        for (int i = 0; i < columns.length; i++) projected[i] = row[columns[i]];
        rowsOut++;
        return projected;
    }

    public void close() { child.close(); LOGGER.debug("columns={} rowsOut={}", columns.length, rowsOut); }
}
```

assemble a new array with
only the necessary columns

Example: DISTINCT Operator Implementation

```
SELECT DISTINCT city FROM trips;
```

```
public final class DistinctOperator implements Operator {  
    private final Operator child;  
    private final Set<List<Object>> seen = new HashSet<>();  
    private int rowsIn, rowsOut;  
  
    public void open() { rowsIn = 0; rowsOut = 0; seen.clear(); child.open(); }  
  
    public Object[] next() {  
        Object[] row = child.next();  
        while (row != null) {  
            rowsIn++;  
            if (seen.add(Arrays.asList(row))) { rowsOut++; return row; }  
            row = child.next();  
        }  
        return null;  
    }  
  
    public void close() { child.close(); LOGGER.debug("rowsIn={} rowsOut={}", rowsIn, rowsOut); }  
}
```

keeps a set of all seen rows

outputs a row only if it is not a duplicate

Example: Sort Operator Implementation

```
SELECT * FROM trips ORDER BY city;
```

```
public final class SortOperator implements Operator {  
    private final Operator child;  
    private final int column;    // the ORDER BY column  
    private Iterator<Object[]> sorted;  
    private int rowsOut;  
  
    public void open() {  
        rowsOut = 0;  
        child.open();  
        List<Object[]> rows = new ArrayList<>();  
        Object[] row = child.next();  
        while (row != null) { rows.add(row); row = child.next(); }  
        rows.sort(Comparator.comparing(r -> (Comparable) r[column]));  
        sorted = rows.iterator();  
    }  
}
```

Reads all rows from the
child operator in open()!
Sorts it here as well.

```
    public Object[] next() {  
        if (!sorted.hasNext()) return null;  
        rowsOut++;  
        return sorted.next();  
    }  
}
```

Returns rows from the
sorted rows it has in
memory.

```
    public void close() { child.close(); LOGGER.debug("column={} rowsOut={}", column, rowsOut); }
```

Quiz Time (2)

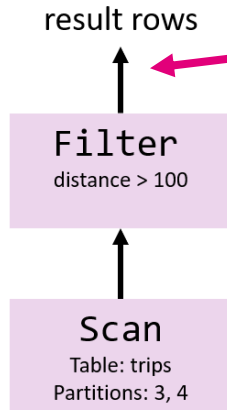
Fill out memory usage and blocking behavior

	Memory Usage	Blocking Behavior
Scan		
Filter		
Project		
Distinct		
Sort		
Max()		

Draining the Tree

- Whoever holds the root drains the whole tree. The root does not know how deep the tree is.
- `close()` in `finally`: the summary log lines are written even when a read fails halfway.
- Week 2's `select(...)` becomes: `plan, then plan.root().rows()`.

```
SELECT * FROM trips WHERE distance > 100;
```



```
default List<Object[]> rows() { // on the Operator interface
    List<Object[]> rows = new ArrayList<>();
    open();
    try {
        for (Object[] row = next(); row != null; row = next()) rows.add(row);
    } finally {
        close();
    }
    return rows;
}
```

Testing an Operator Without a Database

```
final class TestListOperator implements Operator {    // in src/test, not src/main
    private final List<Object[]> rows;
    private Iterator<Object[]> remaining;
    public void open()    { remaining = rows.iterator(); }
    public Object[] next() { return remaining.hasNext() ? remaining.next() : null; }
    public void close()   { }
}
```

Populate rows with whatever data you want to test with

- Create a `TestListOperator` instead of the `ScanOperator`. `TestListOperator` serves rows from memory.
- Then: `FilterOperator` over a `TestListOperator` is a pure unit test: no files, no catalog, no `@TempDir`.
- `ScanOperator` is the one operator that needs files/partitions. Test it with the partitions it is handed, including the empty list.

Stand-Up & Exercise Recap

From Tree to Rows: Executing Queries

- Operator Tree
- Volcano vs. Other Execution Models
- Planning
- Executing

What Volcano Costs

- A virtual call per row per operator: one million rows through three operators is three million `next()` calls.
 - Each call is a branch the CPU cannot predict well and a return value the JIT cannot keep in a register.
 - Tuple-at-a-time interpretation: the operator re-decides what to do for every row.
 - On the JVM: rows are `Object[]`, so every `long` (primitive type) is a boxed `Long` (object), boxing may be overhead.
- Mitigations: vectorized execution, push-based execution + query compilation

Vectorized Execution

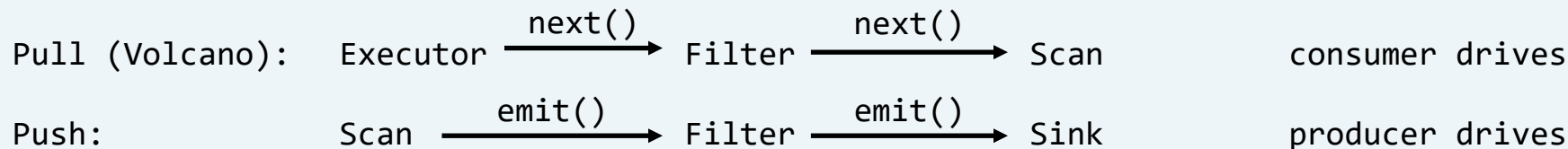
Volcano: `next()` → one row `[Copenhagen | 140 | 210.0]`

Vectorized: `next()` → one batch

city:	<code>[Copenhagen, Aarhus, Aalborg, Esbjerg, ...]</code>
distance:	<code>[140, 187, 210, 299, ...]</code>
price:	<code>[210.0, 301.0, 340.5, 450.25, ...]</code>

- `next()` returns a vector per column instead of one row; the call overhead amortizes over the batch.
- The filter becomes a tight loop over a primitive array, `long[]` not `Long[]`, producing a selection vector.
- DuckDB: vectors of 2048 values flowing through operators, small enough to stay in cache, big enough to amortize.
- What it changes: the operators. What it does not change: the plan shape, the planner, the statistics.

Push-Based Execution



- Producers drive, consumers receive; operators become loops that call downstream.
- DuckDB started pull-based ("Vector Volcano") and moved to push in 2021: parallel execution was hard to express when every operator had to be asked.
- This week's reading: Jaffray, [Query Engines: Push vs. Pull](#). Two short JavaScript programs, same query, both models.

Query Compilation

- HyPer (Neumann, VLDB 2011): generate the tight loop for this query, compile it with LLVM, run it. Interpretation disappears.
- The operator tree still exists at plan time. It just does not exist at run time.
- The trade: compile latency per query, and code that is hard to debug, against the fastest possible per-row path.
- Kersten et al. (PVLDB 2018) measured compiled against vectorized: neither dominates, and both are far ahead of tuple-at-a-time.

Push-Based Execution → Query Compilation

Push-based query compilation results in simpler code and makes unrolling synchronous execution surprisingly easy

```
let result = [];  
Scan(  
  customer,  
  Select(  
    (c) => c.balance > 0,  
    Map(  
      (c) => c.firstName,  
      Distinct((r) => result.push(r))  
    )  
  )  
);  
  
console.log(result);
```



```
let result = [];  
let seen = new Set();  
for (let c of customer) {  
  if (c.balance > 0) {  
    if (!seen.has(c.firstName)) {  
      seen.add(c.firstName);  
      result.push(c.firstName);  
    }  
  }  
}  
  
console.log(result);
```

Comparison

	Pull (Volcano)	Pull (vectorized)	Push (vectorized)	Compiled (push model)
Calls per row	One per operator	One per batch per operator	One per batch per operator	None
Memory per operator	One row	One batch	One batch	Registers
Code complexity	Lowest	Moderate	Moderate	Highest
Driver	Consumer	Consumer	Producer	Producer
Parallelism*	Exchange operator in the plan	Exchange operator in the plan	Morsel-driven, plan unchanged	Morsel-driven, plan unchanged
Stopping early (LIMIT)**	Stop calling next()	Stop calling next()	Signal back up the push chain	Flag tested in the loop

*Pull needs Graefe's exchange operator (SIGMOD 1990) baked into the plan at plan time; push lets any thread grab a morsel and run it through a pipeline (Leis et al., SIGMOD 2014).

**In push the producer owns the loop, so the sink needs a return protocol to say "I am full, stop".

Products (Examples)

	Pull	Push
Row-at-a-time	  SQLite Our engine	(rare)
Vectorized	 Vectorwise  <2021	 today  snowflake
Compiled	(rare)	 Hyper  databricks  Spark  Tungsten

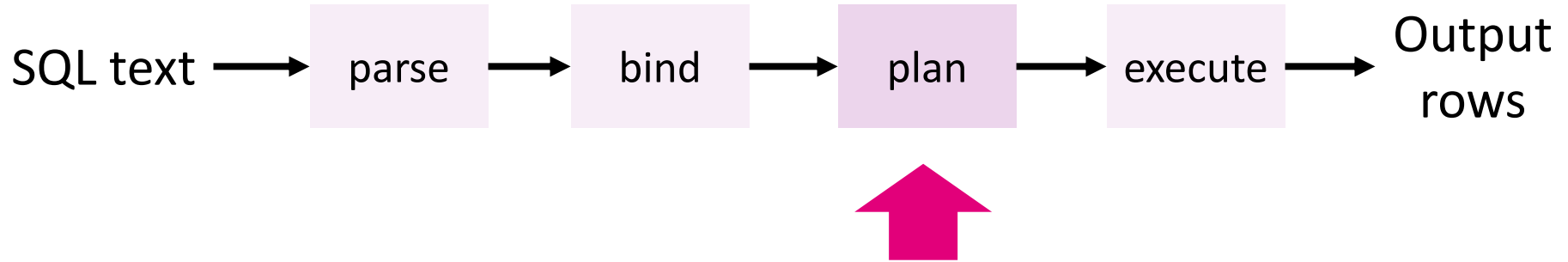
Stand-Up & Exercise Recap

From Tree to Rows: Executing Queries

- Operator Tree
- Volcano vs. Other Execution Models
- Planning
- Executing

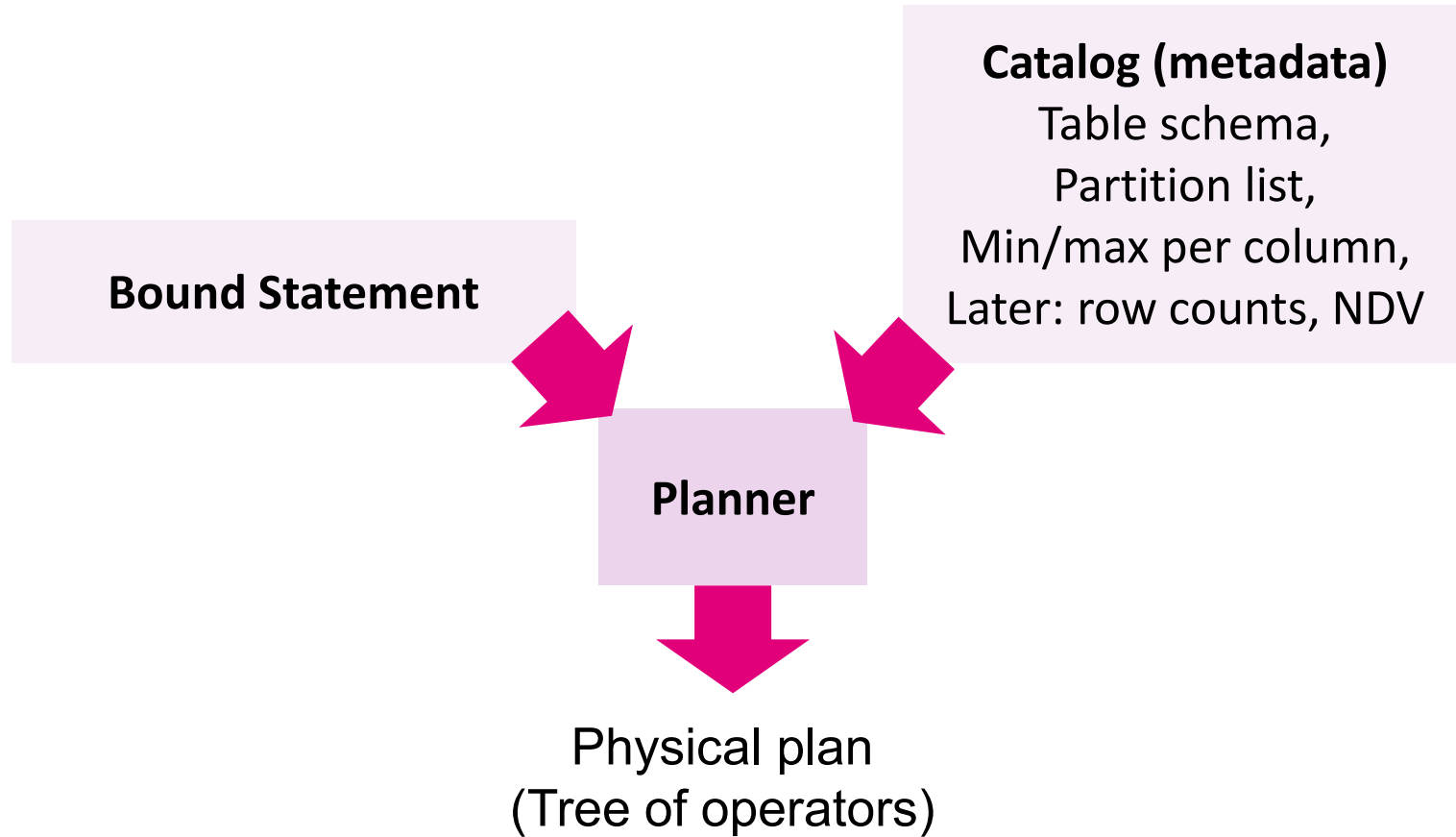
Exercise 4 Preview

We talked about parsing and binding (previous lecture) and execution (previous slides) but forgot the planning step. Somebody has to build the plan: the planner!



Let's talk about the planner!

What a Planner Does (Visual)



What a Planner Does (Textual)

Input: a bound statement, what the query means.

Output: a physical plan, how the engine computes it.

The planner answers the following questions, and answers them from the catalog, never from the data:

1. Which operators, in which tree: the plan's shape.
2. What to read: which partitions, files, or index. Selinger et al. (SIGMOD 1979) called this access path selection. We call it pruning.
3. Which algorithm per operator: hash join or sort-merge, which side builds.
4. Which order: the join order (Part 3), where a filter sits in the tree.
5. How much: estimated rows and cost per alternative, from statistics, which is what decides 2 to 4.

Names differ: PostgreSQL's planner/optimizer picks the cheapest path and builds the plan tree it passes to the executor. DuckDB splits the work into a logical planner, an optimizer, and a physical plan generator. Snowflake calls it compiler.

Planning in Exercise 4

In Exercise 4, planning is two things: build the physical plan, and compute the pruned list of partitions the scan is handed. `Plan(root, scanStats)` is exactly those two, plus the bookkeeping in `scanStats`.

Question	Exercise 4
Which tree	<code>Filter(Scan)</code> , or a bare <code>Scan</code> without a <code>WHERE</code> .
What to read	The pruned partition list, from min/max, handed to the scan.
Which algorithm	Nothing to choose: one scan, one filter.
Which order	One table, no order.
How much	<code>ScanStats</code> : partitions total, read, pruned.

From Bound Statement to Operator Tree

```
SELECT * FROM trips WHERE distance > 100;
```

Bound Statement

`BoundSelect`

`tableName: trips`
`where: distance > 100`



Operator Tree

Filter

`distance > 100`



Scan

`Table: trips`
`Partitions: 2, 3`

Built from the inside out: the scan first, because FROM names what every other clause works on; every further clause wraps the tree built so far.

The root is whatever was wrapped last (Filter in this case), and the root is all the executor holds.

Building the Tree, Inside Out

```
public Plan plan(BoundSelect select) {
    String table = select.tableName();
    Operator root = new ScanOperator(engine, table, ???);           // FROM: which partitions?
    if (select.where().isPresent()) {                                // WHERE: wrap a filter
        root = new FilterOperator(root, ColumnPredicate.of(engine.schema(table), select.where().get()));
    }
    return new Plan(root, ???);                                     // what was decided? ScanStats
}
```

- Two lines per clause: build the operator, make it the new root. An absent clause leaves the tree as it was, so without a **WHERE** the plan is a bare Scan.
- `ColumnPredicate.of(schema, where)` is a record that holds the where clause with additional helper methods (see slide 53).
- A new clause is two more lines and one operator class. The optional column-list extension is exactly that: `root = new ProjectOperator(root, columns)`

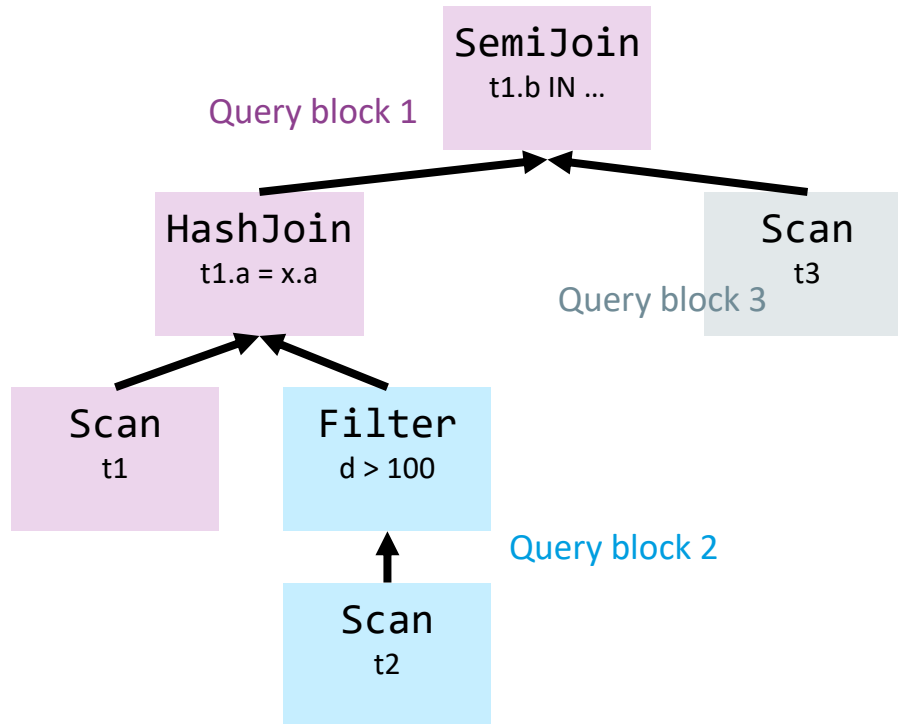
Larger Queries: The Planner Recurses

Query block 1

Query block 2

```
SELECT * FROM t1, (SELECT * FROM t2 WHERE d > 100) x
WHERE t1.a = x.a AND t1.b IN (SELECT b FROM t3);
```

Query block 3



Three query blocks in one statement. `plan(SelectStatement select)` calls itself for every block nested inside, and the result stands where a leaf would.

Where Did the Partition Pruning Go?

Week 2's select, pseudo-code:

```
for each partition p of the table:
    if canMatch(p.min, p.max, comparison, constant): ← needs min/max only
        for each row in read(p): ← needs the data
            if matches(row):
                result.add(row)
```

- Two decisions interleaved in one loop. One needs metadata, one needs data.
- The Volcano scan has no predicate. So the first decision has to happen before the scan exists.

Decide From Metadata, Then Read Data

Partition pruning needs the min/max summaries and never the column data.

→ It belongs at plan time, in the planner, not at run time, in the scan.

The planner's job for `SELECT ... WHERE`:

1. Look up the table's partitions in the catalog.
2. Compare the predicate against each partition's min/max with the helper unit-tested in Exercise 2.
3. Keep the partitions that may contain a match, drop the rest.
4. Build `Filter(Scan(table, survivingPartitions), predicate)`

Without a `WHERE`: all partitions survive, and the plan is a simple `Scan(table, allPartitions)`

Disentangle Min/Max Matching and Row Matching

```
public record ColumnPredicate(int columnIndex, ColumnType type,
                             Comparison comparison, Object constant) {

    /** Whether a partition can hold a matching row. (planner) */
    public boolean canMatch(ColumnRange range) {
        return Values.canMatch(type, range.min(), range.max(), comparison, constant);
    }

    /** Whether the row satisfies the predicate. (filter) */
    public boolean matches(Object[] row) {
        return Values.matches(type, row[columnIndex], comparison, constant);
    }
}
```

Matching min/max range
(coming from the catalog)

Matching a row (coming from
the scan operator)

Planner, Part 1 Version

Builds the operator tree

```
public Plan plan(BoundSelect select) {
    String table = select.tableName();
    int total = engine.partitionCount(table);                // from the catalog, no data read
    List<Integer> partitions = select.where().isPresent()
        ? partitionsToRead(table, ColumnPredicate.of(engine.schema(table), where)) : ...all partitions...

    Operator root = new ScanOperator(engine, table, partitions);    // FROM: the leaf
    if (predicate.isPresent()) {
        root = new FilterOperator(root, predicate.get());          // WHERE: the filter wrapping the scan
    }
    return new Plan(root, new ScanStats(total, partitions.size(), total - partitions.size()));
}
```

```
public List<Integer> partitionsToRead(String table, ColumnPredicate predicate) {
    List<ColumnRange> ranges = engine.columnRanges(table, predicate.columnIndex());    // metadata only
    List<Integer> surviving = new ArrayList<>();
    for (int number = 0; number < ranges.size(); number++) {
        ColumnRange range = ranges.get(number);
        boolean canMatch = predicate.canMatch(range);
        LOGGER.debug("table={} partition={} min={} max={} decision={}", table, number, range.min(),
            range.max(), canMatch ? "READ" : "PRUNED");
        if (canMatch) surviving.add(number);
    }
    return surviving;
}
```

Computes the pruned
partition list

Refactor Visible in the Log

Before (Exercise 2):

```
...,DEBUG,StorageEngine,table=trips column=distance comparison=GREATER_THAN const=100 partition=0 min=12  
max=31 decision=PRUNED  
...,DEBUG,StorageEngine,table=trips column=distance comparison=GREATER_THAN const=100 partition=2 min=140  
max=187 decision=READ
```

After (Exercise 4):

```
...,DEBUG,Planner,table=trips column=distance comparison=GREATER_THAN const=100 partition=0 min=12 max=31  
decision=PRUNED  
...,DEBUG,Planner,table=trips column=distance comparison=GREATER_THAN const=100 partition=2 min=140 max=187  
decision=READ  
...,DEBUG,ScanOperator,table=trips partitions=2 rowsOut=4  
...,DEBUG,FilterOperator,rowsIn=4 rowsOut=4
```

className moved from StorageEngine to Planner; the decision lines now come before any data file is opened.

Stand-Up & Exercise Recap

From Tree to Rows: Executing Queries

- Operator Tree
- Volcano vs. Other Execution Models
- Planning
- Executing

The Executor

```
public void run(String sqlText) {  
    for (Statement statement : new SqlParser().parse(sqlText)) {  
        statementNumber++;  
        MDC.put("statementNumber", String.valueOf(statementNumber));  
        execute(statement);  
    }  
}
```

- The script is parsed once. Then every statement goes through bind, plan, execute, in that order.
- The first error stops the run: an exception leaves the loop.
- Compare to lecture 3's `Binder.bind`: the same switch over the sealed `Statement`, one more time (next slide).



Switch Over Statements

```
private void execute(Statement statement) {
    LOGGER.debug("sql={}", sqlForLog(statement));           // the printer from week 3, commas replaced
    BoundStatement boundStatement = binder.bind(statement);
    switch (boundStatement) {
        case BoundCreateTable create -> {
            engine.createTable(create.tableName(), create.columns());
            LOGGER.debug("statement=CREATE_TABLE table={} durationMs={}", ...);
        }
        case BoundCopy copy -> {
            engine.copyFile(copy.tableName(), copy.csvFilePath());
            LOGGER.debug("statement=COPY table={} durationMs={}", ...);
        }
        case BoundSelect select -> {
            Plan plan = planner.plan(select);
            List<Object[]> rows = plan.root().rows();
            rows.forEach(row -> out.print(toCsv(row) + "\n"));
            LOGGER.debug("statement=SELECT table={} rowsOut={} partitionsRead={} partitionsPruned={} durationMs={}", ...);
        }
    }
}
```

This method drains the tree
(see slide 33)



statementNumber Finally Counts in the Log

```
timestamp,sessionId,statementNumber,threadId,logLevel,className,logMessage
...,3f9c1a2e,0,1,DEBUG,Engine,engine started
...,3f9c1a2e,1,1,DEBUG,Executor,sql=CREATE TABLE trips (city STRING; distance LONG; price DOUBLE);
...,3f9c1a2e,1,1,DEBUG,Executor,statement=CREATE_TABLE table=trips durationMs=4
...,3f9c1a2e,2,1,DEBUG,Executor,sql=COPY trips FROM 'trips.csv';
...,3f9c1a2e,2,1,DEBUG,StorageEngine,table=trips partition=0 column=distance min=12 max=31
...,3f9c1a2e,2,1,DEBUG,Executor,statement=COPY table=trips rows=8 partitions=4 durationMs=11
...,3f9c1a2e,3,1,DEBUG,Executor,sql=SELECT * FROM trips WHERE distance > 100;
...,3f9c1a2e,3,1,DEBUG,Planner,table=trips column=distance ... partition=0 min=12 max=31 decision=PRUNED
...,3f9c1a2e,3,1,DEBUG,ScanOperator,table=trips partitions=2 rowsOut=4
...,3f9c1a2e,3,1,DEBUG,FilterOperator,rowsIn=4 rowsOut=4
...,3f9c1a2e,3,1,DEBUG,Executor,statement=SELECT table=trips rowsOut=4 partitionsRead=2 partitionsPruned=2
durationMs=3
...,3f9c1a2e,0,1,DEBUG,Engine,engine stopped status=0
```

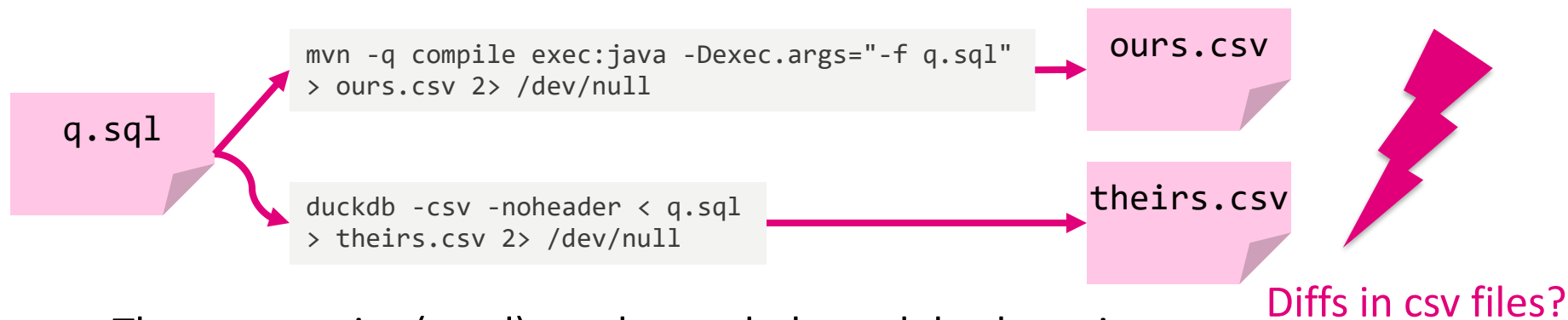
- A session is one run of the engine over one script, one random `sessionId`.
- The executor increments the counter before each statement and puts it in the MDC. First statement is 1. Engine start and stop stay at 0. Put the 0 back when the script is done.
- `WHERE statementNumber = 3` reconstructs the whole story of one statement.
- `WHERE statementNumber > 0` picks out everything statement-related.

Lastly: The “Front Door” of Your Engine

<code>mvn -q compile exec:java</code>	team name and usage
<code>mvn -q compile exec:java -Dexec.args="'SELECT * FROM trips'"</code>	one statement
<code>mvn -q compile exec:java -Dexec.args="-f script.sql"</code>	a whole script

- No arguments: print the team name and usage
- One argument: a SQL query
- Two arguments: `-f` and a path to a SQL file
- Print rows to stdout, console log and errors to stderr
- Data directory: `data/` under the working directory. Add it to `.gitignore`
- Week 5 packages the engine as a JAR with a launcher script, `./engine -f script.sql`

DuckDB as a Differential Oracle



- The same script (`q.sql`), unchanged, through both engines
- Cheap correctness check of your engine
- Known differences:
 - Exponent notation for very large or small doubles (Java `1.0E10` and `1.0E-5`, DuckDB `100000000000.0` and `1e-05`)
 - DuckDB's CLI prints the error and continues with the next statement, your engine stops; both exit with status 1.

Feedback Survey

How is the course so far? Too much, too little, too fast, or too slow? Is it what you expected from the curriculum? Should we repeat anything?



<https://www.menti.com/alsmt9r6awbh>

Summary

Plan vs. execute: decisions that need metadata (pruning) happen in the planner; work that needs data (scanning, filtering) happens in operators.

Volcano: **open**, **next**, **close**; one row per call; null at the end; every operator the same, which is what makes them composable and testable.

**Pull vs. push vs.
materialized***

**Row-at-a-time vs. vectorized
vs. compiled****

The executor numbers statements and stamps the MDC.
The front door prints rows on stdout to diff against DuckDB.

stdout vs. stderr

Foreshadowing Part 3: the planner rearranges plans to optimize join queries.

*See mandatory reading CMU 15-445 notes

**See mandatory reading Jaffray



Image source: [Frie Vilje](#), modified by ChatGPT to "make it dance". Good that the lecture is over.

Appendix: Definition of Checks of Exercise 3

C1 ANTLR in Maven; generated sources not committed.

C2 Required grammar features, including comments, case handling and negative literals.

C3 Requested parser API, statement list and positioned syntax errors.

C4 AST records reuse the engine types.

C5 Catalog-backed binder using schema(...).

C6 Pretty-printer satisfies the round-trip property and has tests.

C7 Default Maven demo prints the four required SQL statements.

C8 Required Week 3 tests are present and green in GitHub CI.

C9 PRs reviewed and merged into protected main.

C10 v0.3 pushed and points to the reviewed commit.

Appendix: The 12 Additional Round-Trip Checks

These checks were written by the reviewer, separately from the student tests. Each parses SQL into an AST, prints the AST as SQL, parses the output, and compares the two ASTs for equality. The printed output is tested directly, without adding a semicolon.

Twelve is the number of cases chosen for this review, not an Exercise 3 requirement or a grade. The exercise requires coverage of every statement shape; the extra cases check value preservation and formatting boundaries.

No.	What is checked	SQL input
1	CREATE TABLE	CREATE TABLE trips (city STRING, distance LONG, price DOUBLE);
2	COPY	COPY trips FROM 'trips.csv';
3	SELECT without WHERE	SELECT * FROM trips;
4	Positive integer	SELECT * FROM trips WHERE distance > 100;
5	Negative integer	SELECT * FROM trips WHERE distance = -1;
6	Negative decimal	SELECT * FROM trips WHERE price = -1.5;
7	String literal	SELECT * FROM trips WHERE city = '12';
8	Keyword and identifier casing	select * from Trip_Data9;
9	Comment and whitespace	-- comment [newline] SELECT * FROM trips;
10	Large decimal	SELECT * FROM trips WHERE price = 10000000.0;
11	Small decimal	SELECT * FROM trips WHERE price = 0.00001;
12	Negative zero	SELECT * FROM trips WHERE price = -0.0;

In case 9, [newline] represents an actual line break. A score of 10/12 means ten of these cases passed and two failed. These checks do not execute SQL or test the binder.